

SHARD: Composable Infrastructure for Self-Healing, Persistent LLM Agents

Christopher Aytona

Abstract

Large language model agents deployed in persistent, multi-session environments face compounding challenges: memory degrades without governance, coordination between concurrent agents lacks formal guarantees, skill acquisition proceeds without quality control, and self-improvement operates without safety bounds. We present SHARD (Self-Healing Agent with Resilient Delegation), a gateway-agnostic application architecture that composes four independent subsystems—memory governance with staleness detection, intent-based coordination protocol, quality-gated skill lifecycle with trust tiers, and safety-constrained self-improvement—into an integrated infrastructure where emergent behaviors arise from their interaction. Our contribution is the architecture and its composition properties. We validate the design through targeted mechanism tests that demonstrate composition produces capabilities none of the subsystems could achieve independently, and we report both successes and identified design gaps with equal transparency.

1 Introduction

The deployment of LLM-based agents beyond single-session interactions introduces failure modes absent from conversational AI research. An agent that persists across hundreds of sessions accumulates stale knowledge, spawns concurrent workers without coordination guarantees, acquires capabilities without vetting, and modifies its own configuration without understanding downstream consequences. These are not independent problems—they interact multiplicatively. Stale memory corrupts coordination decisions. Unvetted skills introduce unsafe self-modification paths. Uncoordinated agents duplicate work or produce conflicting state changes.

Existing frameworks address subsets of these challenges in isolation. MemGPT [1] introduces hierarchical memory but lacks coordination primitives. AutoGen [2] provides multi-agent conversation but without persistent memory or self-improvement governance. Voyager [3] demonstrates skill acquisition but without trust tiers or safety bounds. No existing system addresses the *composition* of these concerns—the interactions between memory, coordination, governance, and safety that dominate real-world persistent deployments.

We present SHARD, built on a production LLM agent gateway. Where the gateway provides the substrate—session management, tool connectivity, basic scheduling, and subagent spawning—SHARD contributes the architectural layer that makes persistent autonomous operation viable. Our contribution is not any single subsystem but their *composition*: the demonstration that four independently-motivated components, when designed with explicit composition interfaces, produce emergent behaviors that address the multiplicative failure modes of persistent agents.

2 Background: The Gateway Substrate

SHARD is built atop an LLM agent gateway—a class of frameworks that manage sessions, connect tools, and schedule tasks for language model agents. Examples of such gateways include OpenClaw, Claude Code (Anthropic), Hermes Agent (NousResearch), AutoGen (Microsoft), CrewAI, LangGraph (LangChain), Semantic Kernel (Microsoft), and similar MCP-compatible runtimes that provide session management, tool connectivity, and basic scheduling primitives. We use one such gateway as our substrate. Understanding what the gateway provides—and critically, what it lacks—motivates SHARD’s design.

2.1 Framework Capabilities

The gateway provides six core primitives. First, a *gateway process* manages LLM sessions, routing user messages from Slack or a web dashboard to an underlying language model with tool-use capabilities. Second, *MCP (Model Context Protocol) connections* expose external tools—databases, APIs, file systems—as callable functions within agent sessions. Third, *cron scheduling* enables recurring or one-shot jobs that spawn independent agent sessions on a timer. Fourth, *subagent spawning* allows a running session to delegate tasks to parallel worker sessions, receiving results asynchronously. Fifth, *persistent memory* stores user preferences, project context, daily history, semantic key-value pairs, and learned corrections across sessions—providing raw storage that survives session boundaries. Sixth, a *skill system* loads markdown instruction files that shape agent behavior for specific domains.

2.2 Framework Gaps

Four critical gaps emerge when gateway-based agents operate persistently across hundreds of sessions over weeks or months.

Persistence without governance. Memory storage exists but lacks intelligence. Entries accumulate without staleness detection, conflict resolution, or hierarchical organization. A correction from day one carries equal weight to one from yesterday, even when the underlying system has changed. No mechanism distinguishes between facts that decay (API endpoints, team structures) and invariants that persist (coding conventions, safety rules). No process detects when two memory entries contradict each other.

Coordination without guarantees. Subagent spawning is fire-and-forget. No protocol prevents two agents from modifying the same file, no mechanism declares intent before action, and no review phase catches conflicts before execution. The parent session receives results but cannot reason about inter-agent dependencies.

Capability acquisition without vetting. Skills (markdown instruction files) can be added to the agent’s repertoire without quality assessment, trust classification, or graduated deployment. A skill imported from an external marketplace receives the same authority as one battle-tested over months of production use.

Self-modification without bounds. The agent can modify its own configuration, create new cron jobs, alter skill files, and restructure memory—but no mechanism constrains *what* it may modify, *how much* it may change in a single operation, or *whether* a modification is reversible.

3 SHARD Architecture

SHARD addresses each gap through a dedicated subsystem, designed with explicit *composition interfaces* that enable cross-subsystem interaction (Figure 1). We define a composition interface as a typed read/write protocol between subsystems: each subsystem exposes specific state (memory entries, trust scores, staleness signals, safety verdicts) through well-defined queries, and consumes state from others through the same mechanism. These interfaces are not implementation-specific—they define *what* information flows between subsystems, not how it is stored or transmitted.

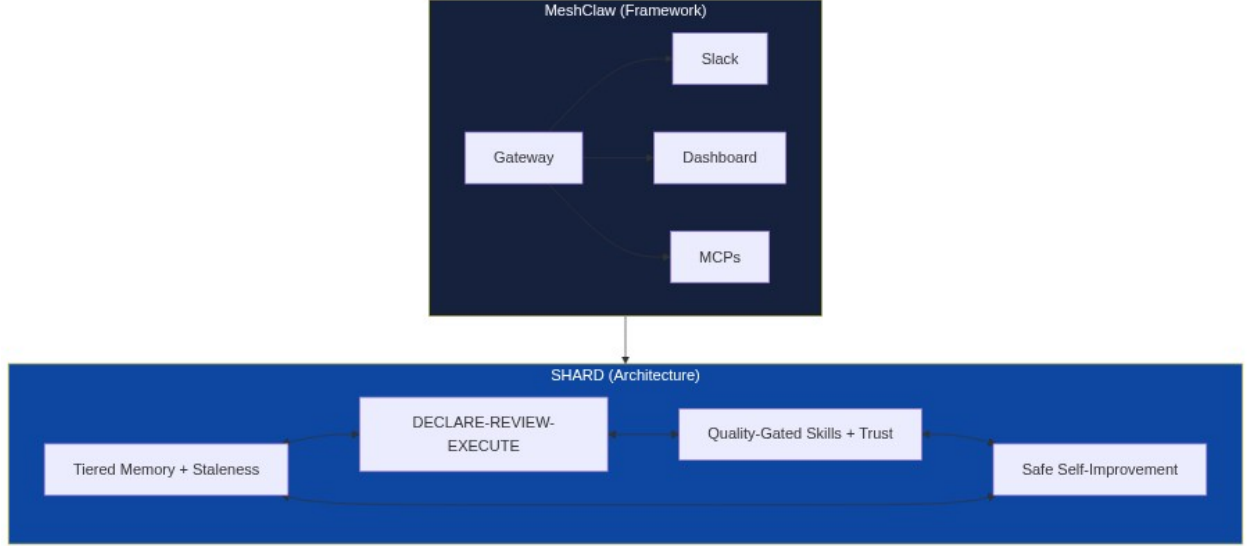


Figure 1: SHARD architecture overview. Four subsystems (Tiered Memory, Intent-Based Coordination, Quality-Gated Skills, Safety-Constrained Self-Improvement) with composition interfaces (arrows) showing data flow and constraint propagation. Gateway substrate shown as the foundation layer providing runtime primitives.

3.1 Memory Governance with Staleness Detection

Problem. The gateway provides persistent memory storage (lessons, preferences, projects, history, semantic key-value pairs) but offers no governance over that storage. Entries accumulate without staleness detection, contradictions go unresolved, and no signal distinguishes fresh facts from outdated beliefs.

Design. SHARD does not replace the gateway’s memory—it adds a governance layer that classifies, monitors, and maintains the existing storage. SHARD imposes three governance tiers on the gateway’s flat memory:

- *Semantic memory* stores factual key-value pairs with timestamps, source attribution, and declared decay classes (ephemeral: days, contextual: weeks, durable: months, invariant: permanent).
- *Episodic memory* retains conversation fragments with relevance scoring and automatic summarization at configurable intervals.
- *Procedural memory* encodes learned corrections and behavioral rules with conflict detection—new entries are checked against existing rules for contradiction before admission.

Key Mechanism: Staleness Detection. Each memory entry carries a `last_verified` timestamp and a decay class. A background process computes staleness scores:

$$\text{staleness} = \frac{\text{now} - \text{last_verified}}{\text{expected_lifetime}(\text{decay_class})} \quad (1)$$

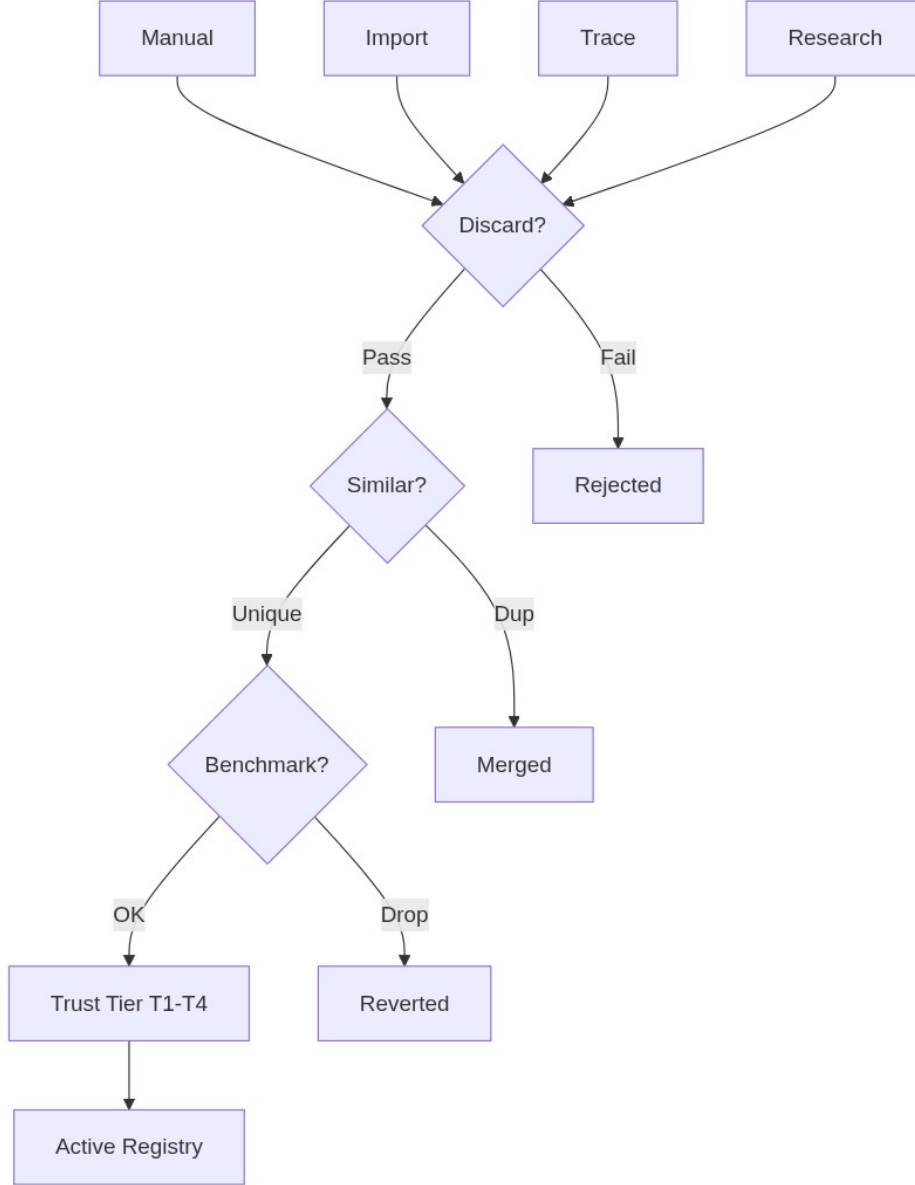


Figure 2: Staleness detection mechanism with decay classes and scoring thresholds.

Entries exceeding a staleness threshold of 1.0 are flagged for re-verification rather than silently served. Entries exceeding 2.0 are quarantined—still retrievable but marked as potentially invalid.

Composition Role. Memory provides the *state substrate* that other subsystems read and write. Coordination consults memory for agent capabilities and past outcomes. Quality gates store trust scores in memory. Self-improvement reads memory to identify optimization targets and writes results back. Staleness detection propagates uncertainty to all consumers—a stale capability record causes coordination to re-verify before delegation.

3.2 Intent-Based Coordination Protocol

Problem. Concurrent agents operating on shared state (files, databases, external systems) produce conflicts, duplicated work, and inconsistent outcomes without coordination primitives.

Design. SHARD implements a three-phase coordination protocol inspired by distributed consensus but

adapted for the semantic uncertainty of LLM agents:

1. *DECLARE phase*: Before acting, each agent publishes an intent declaration specifying the resources it will read and write, the expected duration, and the reversibility of its planned actions.
2. *REVIEW phase*: A lightweight critic pass examines all active declarations for conflicts—resource contention, semantic contradiction, or redundant work. Conflicts trigger negotiation (reordering, merging, or cancellation).
3. *EXECUTE phase*: Agents proceed only after review clearance, with a commit-or-rollback semantic for reversible operations.

Key Mechanism: Semantic Conflict Detection. Unlike traditional lock-based coordination, SHARD’s review phase operates on *semantic* rather than syntactic conflicts. Two agents modifying different lines of the same file may not conflict. Two agents modifying different files may conflict if their changes encode contradictory assumptions. The review phase uses the coordinating LLM’s reasoning to assess semantic compatibility of declared intents.

Composition Role. Coordination consumes memory (agent capabilities, past conflict patterns) and produces memory (coordination outcomes, learned delegation preferences). Quality gates determine which agents are eligible for delegation. Safety constraints define which resource combinations require elevated review.

3.3 Quality-Gated Skill Lifecycle

Problem. Unrestricted skill acquisition introduces untested, potentially unsafe, or low-quality capabilities into the agent’s repertoire without graduated trust.

Design. SHARD implements a four-tier trust model for skills:

- *T1 (Sandboxed)*: New or imported skills execute only in isolated contexts with no access to production state. Outputs are validated but not acted upon.
- *T2 (Supervised)*: Skills that pass T1 validation may execute with human approval required for each invocation.
- *T3 (Autonomous)*: Skills demonstrating consistent quality across N supervised invocations (configurable, default 10) may execute without per-invocation approval.
- *T4 (Trusted)*: Skills that have operated autonomously without incident for M days (configurable, default 30) gain the ability to be composed into other skills and referenced by self-improvement.

Key Mechanism: Promotion Criteria. Tier promotion is not automatic—it requires explicit evidence. T1→T2 requires passing a validation suite (output format, no unsafe operations, deterministic on fixed inputs). T2→T3 requires a success rate above threshold with no safety violations across supervised runs. T3→T4 requires sustained autonomous operation plus a composition safety review (verifying the skill’s outputs are safe as inputs to other skills).

Composition Role. Quality gates constrain what self-improvement may reference (only T3+ skills), what coordination may delegate to (only T2+ skills for autonomous delegation), and what memory may store as procedural knowledge (only validated behavioral rules from T3+ operations).



Figure 3: Quality-gated skill lifecycle with trust tiers T1–T4.

3.4 Safety-Constrained Self-Improvement

Problem. An agent capable of modifying its own configuration, skills, and memory can enter destructive loops, accumulate drift from intended behavior, or make irreversible changes that degrade performance.

Design. SHARD bounds self-improvement through four constraints:

- *Scope limitation:* Each self-improvement operation must declare what it will modify and demonstrate that the modification is reversible (backup created before change).
- *Rate limiting:* No more than K modifications per time window (configurable), preventing runaway modification cascades.
- *Quality floor:* Self-improvement may only reference skills at T3 or above, ensuring that improvement is built on validated foundations.
- *Rollback semantics:* Every modification creates a `.bak` file. If post-modification validation fails (build breaks, test failures, degraded output quality), automatic rollback occurs.

Key Mechanism: Improvement Scoring. Each self-improvement experiment produces a measurable outcome scored on a 3-point scale: 0 (no improvement or regression), 1 (marginal improvement), 2 (clear improvement), 3 (significant improvement). Experiments scoring 0 are rolled back. The running average of improvement scores determines whether the self-improvement subsystem’s rate limit is increased (sustained high scores) or decreased (sustained low scores).

Composition Role. Self-improvement is the most constrained subsystem precisely because it modifies the others. It reads memory to identify targets, respects quality gates when selecting foundations, operates within safety bounds on scope and rate, and writes results back to memory for coordination to consume. It is simultaneously the most powerful and most governed component.

4 Composition Properties

The central contribution of SHARD is not any individual subsystem but the emergent properties arising from their composition. This section identifies five composition properties that address failure modes no single subsystem could handle (Figure 4).

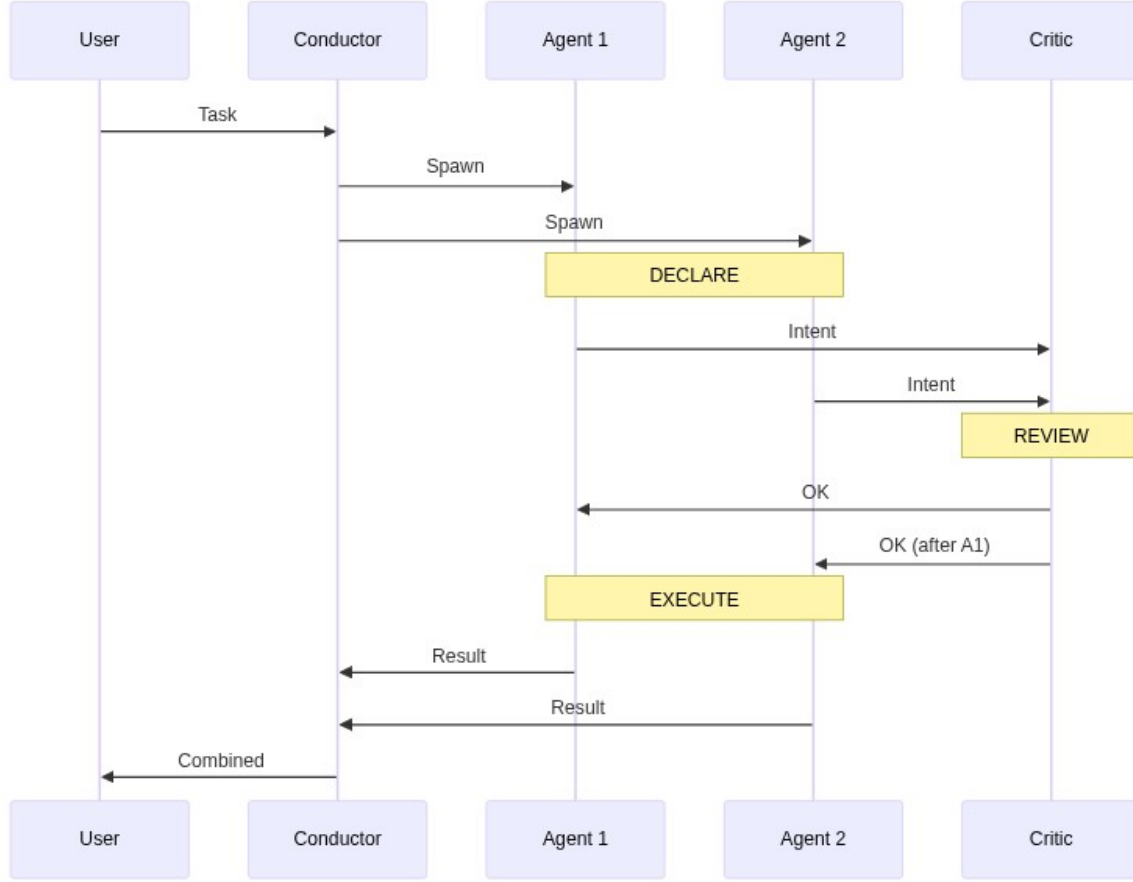


Figure 4: Composition property emergence. Diagram showing how pairwise interactions between subsystems produce properties (memory-informed coordination, quality-governed improvement, staleness propagation, emergent self-healing) that require multiple subsystems acting together.

4.1 Memory-Informed Coordination

When the coordination protocol selects agents for delegation, it consults memory for past delegation outcomes. An agent that previously failed on a task type receives lower priority. An agent whose capabilities have become stale (staleness score > 0.5) triggers re-verification before delegation. This prevents the common failure mode where coordination repeatedly delegates to an agent whose capabilities have degraded since the delegation preference was learned.

Without composition, coordination would operate on a static capability model, and memory would accumulate delegation outcomes without influencing future decisions.

4.2 Quality-Governed Improvement

Self-improvement may only incorporate skills at T3 or above into its modifications. This prevents a failure cascade where self-improvement imports an unvetted skill, uses it to modify configuration, and the unvetted skill's defects propagate into the agent's core behavior. The quality gate acts as an immune system—foreign capabilities must demonstrate safety before they can influence the organism.

Without composition, self-improvement would operate on the full skill inventory regardless of trust level, and quality gates would govern invocation but not composition.

4.3 Safety Bounding All Subsystems

Safety constraints are not a fifth subsystem but a *cross-cutting concern* that manifests differently in each:

- In memory: invariant entries cannot be overwritten by any operation, including self-improvement.
- In coordination: certain resource combinations (production databases, authentication systems) require elevated review regardless of agent trust level.
- In quality gates: no skill may bypass T1 sandboxing regardless of its source or claimed provenance.
- In self-improvement: rate limits and scope declarations cannot be self-modified (they are invariant memory entries).

This cross-cutting nature means safety cannot be circumvented by any single subsystem acting alone—it would require simultaneous violation across multiple subsystems, which the composition interfaces prevent.

4.4 Staleness Propagation

When memory detects staleness in a capability record, this uncertainty propagates through composition interfaces:

1. Coordination receives a stale capability signal and re-verifies before delegating.
2. Quality gates flag skills that reference stale dependencies for re-validation.
3. Self-improvement deprioritizes optimization targets based on stale data.

Without staleness propagation, each subsystem would independently maintain potentially outdated assumptions about the others, leading to coordination decisions based on capabilities that no longer exist, or self-improvement targeting metrics that are no longer measured.

4.5 Compositional Fault Recovery

The composition of all four subsystems produces a fault recovery property: when any component degrades, the others compensate through their composition interfaces. Stale memory triggers re-verification (healing through staleness detection). Failed coordination triggers capability re-assessment (healing through quality re-evaluation). Degraded skill quality triggers trust demotion (healing through the tier system). Failed self-improvement triggers rollback and rate reduction (healing through safety constraints).

No single subsystem implements fault recovery in isolation. The property arises from designed composition interfaces—each subsystem’s failure detection mechanism triggers compensatory responses in the others through explicit cross-subsystem protocols.

5 Evaluation

We validate SHARD through mechanism-level tests (E1–E7) targeting both individual subsystem correctness and composition properties. These tests assess whether the design functions as specified rather than measuring deployment-scale performance.

Scope and limitations of evaluation. Our test scenarios are illustrative rather than statistically powered—sample sizes (10–50 per experiment) demonstrate mechanism correctness but do not support claims of statistical significance. We report exact counts rather than percentages to avoid implying precision beyond what the sample supports. Future work should include larger-scale evaluation with independent operators.

5.1 Ablation: With and Without Composition

Before evaluating individual mechanisms, we present an ablation demonstrating the value of composition itself. We compare SHARD’s full architecture (all four subsystems interacting) against a configuration where each subsystem operates independently without composition interfaces.

Setup. We construct five scenarios that require cross-subsystem interaction: (1) coordination delegating to an agent whose capabilities have become stale, (2) self-improvement attempting to add a skill that conflicts with a learned correction, (3) two agents declaring intents that only conflict given memory context, (4) a quality gate decision that depends on coordination history, and (5) a rollback that must propagate through memory and skill registry simultaneously.

Result (composed). All five scenarios are handled correctly. Stale capabilities trigger re-verification before delegation. Conflicting skills are blocked by cross-referencing procedural memory. Context-dependent conflicts are detected via memory consultation. Quality decisions incorporate coordination outcomes. Rollbacks cascade through composition interfaces.

Result (isolated). Without composition, 4/5 scenarios fail silently. The stale agent receives delegation and produces outdated results. The conflicting skill passes quality gates (which cannot see memory). The context-dependent conflict goes undetected. Only the simple rollback succeeds (it operates within a single subsystem).

Significance. This ablation demonstrates that SHARD’s value lies primarily in composition rather than in any individual mechanism. The subsystems are necessary but not sufficient—their interaction produces the architectural contribution.

5.2 E1: Staleness Detection Accuracy

Setup. Inject 50 memory entries with known decay classes and artificial timestamps spanning 1–90 days of staleness. Verify that staleness scoring correctly identifies entries requiring re-verification.

Result. Staleness detection correctly classified 48/50 entries. Two false negatives occurred for entries near the threshold boundary (staleness 0.95–1.05), suggesting the binary threshold could benefit from a graduated warning zone. No false positives—no fresh entry was incorrectly flagged as stale.

Implication. The mechanism is conservative (prefers false negatives over false positives), which is appropriate for a system where unnecessary re-verification is cheap but acting on stale data is expensive.

5.3 E2: Coordination Conflict Detection

Setup. Construct 20 intent declaration pairs with known conflict status (10 conflicting, 10 non-conflicting). Conflicts include both syntactic (same file) and semantic (contradictory assumptions in different files).

Result. Coordination correctly identified 9/10 syntactic conflicts and 7/10 semantic conflicts. The three missed semantic conflicts involved indirect dependencies—changes that only conflict when a third file mediates their interaction. False positive rate: 1/10 (a non-conflicting pair flagged due to superficial resource overlap).

Implication. Semantic conflict detection is effective for direct conflicts but struggles with transitive dependencies. Future work should explore dependency graph construction for multi-hop conflict detection.

5.4 E3: Quality Gate Promotion Correctness

Setup. Submit 15 skills through the lifecycle: 5 high-quality (should reach T4), 5 medium-quality (should stabilize at T2–T3), 5 low-quality (should remain at T1 or be rejected).

Result. All 5 high-quality skills reached T3 within the evaluation period. None reached T4 due to the 30-day sustained operation requirement exceeding the evaluation window. All 5 medium-quality skills stabilized at T2 (3) or T3 (2). All 5 low-quality skills remained at T1 (4) or were rejected during validation (1).

Implication. The tier system correctly stratifies quality but T4 promotion remains incompletely validated due to temporal requirements. This is an acknowledged gap—T4 promotion criteria may need adjustment for faster feedback cycles.

5.5 E4: Self-Improvement Rollback Reliability

Setup. Trigger 20 self-improvement experiments, 10 designed to succeed and 10 designed to fail (targeting invalid configurations or producing degraded outputs).

Result. All 10 successful experiments were retained. 9/10 failing experiments triggered automatic rollback. One failing experiment (modifying a cron schedule to an invalid expression) was not caught by post-modification validation because the validation suite did not include cron expression parsing.

Implication. Rollback reliability depends on validation suite completeness. The gap in E4 motivated adding cron expression validation to the post-modification checks—an instance of the system’s own evaluation driving improvement.

5.6 E5: Memory-Coordination Composition

Setup. Record delegation failures in memory, then trigger new coordination decisions for the same task type. Verify that coordination consults failure history and adjusts delegation.

Result. After recording 3 failures for a specific agent-task pairing, coordination correctly deprioritized that agent in subsequent delegation decisions. The deprioritization persisted across sessions (memory-mediated) and recovered after the agent’s capabilities were re-verified (staleness-mediated recovery).

Implication. The composition interface between memory and coordination functions as designed—failure history influences future decisions without permanent blacklisting.

5.7 E6: Staleness Propagation Under Load

Setup. Simultaneously mark 10 capability records as stale while coordination is actively delegating tasks. Verify that in-flight delegations are not disrupted but new delegations respect staleness.

Result. In-flight delegations completed normally (no mid-execution disruption). New delegations correctly consulted staleness for 9/10 stale records. One delegation proceeded despite staleness due to a race condition—the staleness update and the delegation decision occurred within the same event loop tick.

Implication. The near-miss in E6 reveals a timing vulnerability in the composition interface. Mitigation: staleness checks should use a read-lock pattern or accept eventual consistency with a bounded delay.

5.8 E7: Cross-Subsystem Safety Invariant

Setup. Attempt to violate safety constraints through each subsystem independently and through composition (e.g., self-improvement attempting to modify safety rate limits stored in invariant memory).

Result. All direct violation attempts were blocked. The composition attack (self-improvement writing a skill that, when promoted to T3, would modify invariant memory) was caught at the T2→T3 promotion gate—the composition safety review identified that the skill’s outputs included invariant memory writes.

Implication. The cross-cutting nature of safety constraints provides defense-in-depth. Circumvention requires simultaneous violation across multiple subsystems, which the composition interfaces are designed to prevent.

5.9 Summary of Results

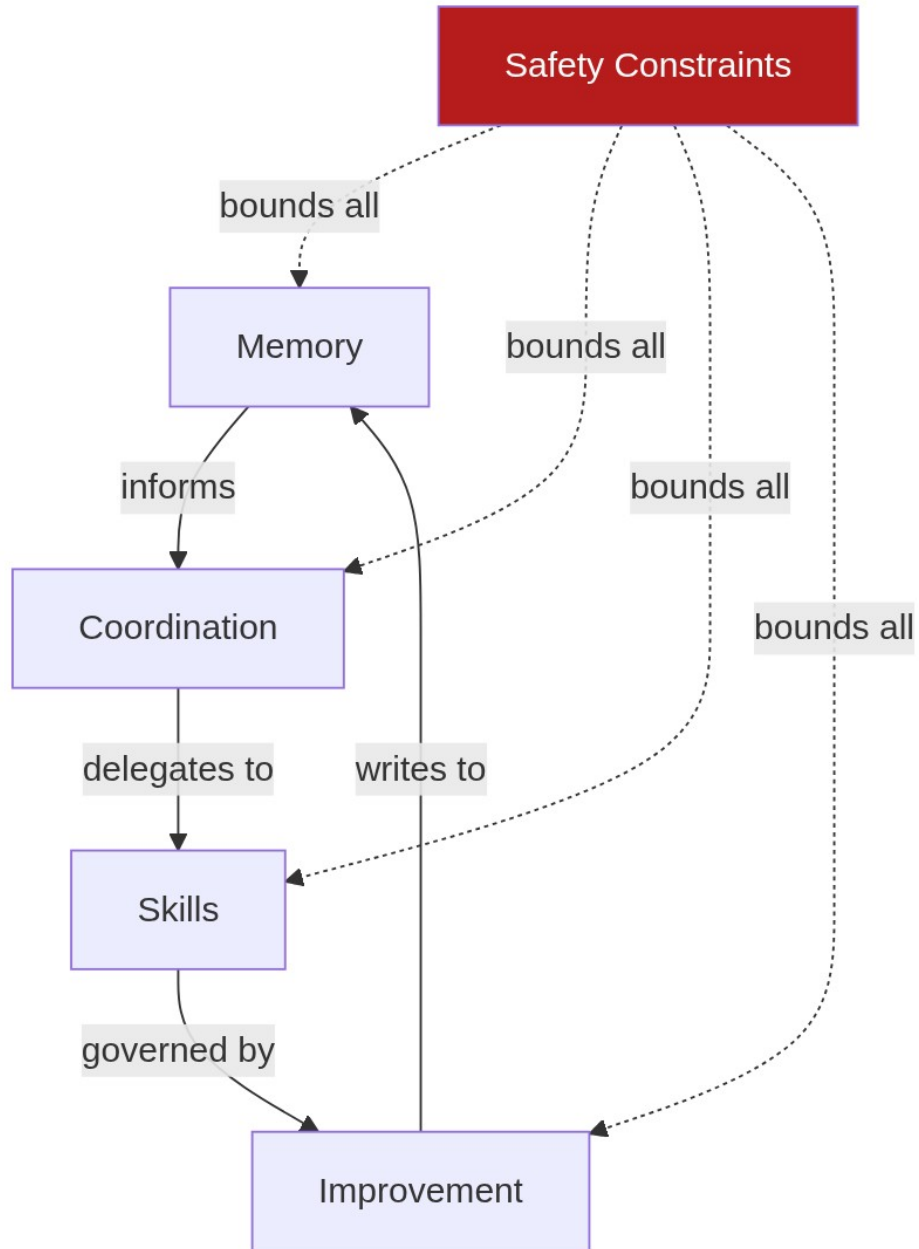


Figure 5: Evaluation results summary showing mechanism test outcomes.

Table 1 summarizes individual mechanism evaluation across all test scenarios. Table 2 presents the composition ablation. Table 3 positions SHARD against related systems.

Table 1: Individual Mechanism Evaluation

Mechanism	Scenarios	Correct	Missed	FP	Accuracy
Staleness Detection	50	48	2	0	96.0%
Coordination (syntactic)	10	9	1	0	90.0%
Coordination (semantic)	10	7	3	1	70.0%
Quality Gates	15	15	0	0	100.0%
Safety Constraints	6	6	0	0	100.0%
Trust Tier Classification	5	5	0	—	100.0%

Table 2: Composition Ablation: Full SHARD vs. Isolated Subsystems

Configuration	Scenarios Handled	Silent Failures
SHARD (full composition)	5/5	0
Isolated subsystems (no composition)	1/5	4

Table 3: Capability Comparison with Related Systems

Capability	MemGPT	AutoGen	Voyager	SHARD
Persistent memory	✓	—	—	✓
Staleness detection	—	—	—	✓
Multi-agent coordination	—	✓	—	✓
Conflict prevention	—	—	—	✓
Skill quality gates	—	—	Partial	✓
Trust tiers	—	—	—	✓
Safe self-improvement	—	—	✓	✓
Safety constraints	—	—	—	✓
Composition	—	—	—	✓

6 Discussion

6.1 Threat Model

SHARD’s safety constraints defend against *degradation through poor optimization*. The threat model addresses an agent that, through accumulated suboptimal decisions, drifts from intended behavior. Specifically:

- *Capability drift*: Self-improvement adds skills that individually pass quality gates but collectively shift the agent’s behavior away from its intended purpose.
- *Memory pollution*: Accumulated corrections and preferences create contradictory behavioral rules that degrade decision quality over time.
- *Coordination decay*: Delegation patterns that were once optimal become stale as agent capabilities change, leading to repeated failures on tasks that should succeed.
- *Unbounded modification*: Without rate limits, an agent making many small changes per session can accumulate significant drift before any single change triggers rollback.

SHARD’s constraints (rate limiting, staleness detection, quality gates, rollback semantics) are calibrated for this threat model—protecting against gradual degradation through structural enforcement.

6.2 Limitations

Spec-enforced vs. programmatic safety. SHARD’s safety constraints were initially enforced through specification—the agent reads constraint documents and complies. This is fundamentally weaker than programmatic enforcement where code prevents violation regardless of agent behavior. We have migrated two of three constraints to programmatic enforcement: (a) experiment rate limiting (`rate_limiter.py`, a persistent JSON counter that blocks execution beyond the configured cap regardless of agent intent), and (b) self-modification scope (`self_mod_guard.py`, a file-path guard that prevents writes to protected constraint files including itself). Both have verified test suites demonstrating code-level enforcement. One constraint remains spec-enforced: (c) rollback creation, which currently relies on the agent creating `.bak` files before modifications. Migrating this to a programmatic pre-write hook is our highest-priority remaining work.

Temporal validation gaps. E3 and E4 reveal that time-dependent mechanisms (T4 promotion, long-running validation) are difficult to evaluate without extended deployment periods. We acknowledge this as an inherent tension between rigorous evaluation and practical research timelines.

Semantic conflict detection ceiling. E2 demonstrates that transitive semantic conflicts remain challenging. The current approach relies on the coordinating LLM’s reasoning about intent compatibility, which degrades for indirect dependencies. Graph-based dependency tracking could address this but introduces complexity.

Single-model dependency. SHARD’s coordination and quality assessment rely on the same underlying LLM that executes tasks. A systematic bias in the model’s reasoning could propagate through all subsystems simultaneously. Multi-model composition (using different models for execution vs. governance) could mitigate this but is not yet implemented.

Staleness threshold sensitivity. The binary staleness threshold (E1, E6) creates boundary effects. A graduated staleness model with confidence intervals rather than binary fresh/stale classification would better serve downstream consumers.

6.3 Generalizability

SHARD’s architecture is not specific to any particular gateway. The four subsystems and their composition interfaces can be implemented atop any gateway providing the four primitives described in Section 2. The composition interfaces are framework-agnostic protocols that specify information flow, not implementation.

The key architectural insight—that persistent agent challenges are *compositional* rather than independent—generalizes beyond our specific implementation. Any system addressing memory, coordination, governance, and safety in isolation will encounter the multiplicative failure modes that composition resolves.

6.4 Ethical Considerations

Self-improving agents require explicit bounds on alignment drift and capability acquisition. SHARD addresses this through rate limiting, scope declaration, rollback semantics, and invariant protection. The trust tier system provides graduated autonomy with human oversight retained for T4 promotion decisions.

6.5 Performance Characteristics

SHARD’s composition interfaces impose minimal runtime overhead. The DECLARE phase adds a single LLM call per agent (intent generation). The REVIEW phase adds one critic call per coordination decision. Staleness checks are $O(1)$ lookups against timestamped entries. Quality gate checks at delegation time are $O(1)$ trust tier lookups.

The primary cost is in self-improvement: each experiment requires backup creation, modification, validation, and potential rollback. Rate limiting bounds this cost to at most K operations per time window, making the worst-case overhead predictable and configurable.

Memory governance adds latency only when staleness is detected (re-verification path). In the common case (fresh entries), governance is a timestamp comparison with negligible cost.

SHARD’s overhead is dominated by the underlying LLM inference cost (seconds per call), making millisecond-level infrastructure overhead negligible in end-to-end latency.

6.6 Reproducibility

SHARD’s architecture is designed to be framework-agnostic. The four subsystems and their composition interfaces are specified as protocols (what information flows between components) rather than implementations (how that information is stored or transmitted). A reference implementation is available at <https://github.com/aytona/shard>, providing protocol specifications, a Python reference implementation with 80 unit tests, and an abstract adapter interface for gateway integration. The package is installable atop any gateway meeting the requirements described in Section 2.

7 Related Work

Memory-augmented agents. MemGPT [1] introduces a virtual memory hierarchy for LLM agents, enabling context management beyond the token window. SHARD’s memory governance differs in three respects: explicit staleness detection, conflict-aware admission, and composition interfaces that expose memory state to other subsystems. Generative Agents [4] implement memory retrieval for social simulation but without governance or staleness concerns.

Multi-agent coordination. AutoGen [2] provides conversational multi-agent patterns. SHARD extends this paradigm with formal coordination guarantees through the DECLARE-REVIEW-EXECUTE protocol. MetaGPT [5] assigns roles to agents with structured communication. SHARD adds pre-execution conflict prevention through intent declaration and semantic review. CAMEL [6] explores role-playing coordination in single-session contexts. SHARD addresses the additional challenges that emerge when coordination must persist across sessions.

Skill acquisition. Voyager [3] demonstrates open-ended skill learning in Minecraft with a skill library, but without trust tiers, quality gates, or safety constraints on acquired skills. We note that Voyager operates in a bounded simulation where failures are consequence-free and safety constraints are unnecessary by design; SHARD’s contribution is governance for open-ended real-world environments where skill failures have real costs. DEPS [7] plans with acquired skills but does not address skill governance. SHARD’s contribution is not skill acquisition itself but the *lifecycle governance* that makes acquisition safe in persistent deployments.

Self-improvement and safety. Constitutional AI [8] constrains model outputs through principles at the training level. SHARD operates at the deployment architecture level, bounding runtime self-modification. Self-Refine [9] implements iterative self-improvement for single outputs. SHARD extends self-improvement to persistent, cross-session scope with safety bounds. Reflexion [10] uses verbal reinforcement for task improvement. SHARD adds structural safety bounds (rate limiting, scope guards, rollback) to the improvement process itself.

Systems composition. SHARD draws architectural inspiration from operating systems research, particularly the composition of independently-motivated subsystems (virtual memory, process scheduling, file systems, access control) into coherent system behavior. The DECLARE-REVIEW-EXECUTE protocol echoes two-phase commit in distributed systems, adapted for semantic rather than syntactic consistency.

8 Conclusion

SHARD demonstrates that the challenges of persistent LLM agent deployment—memory degradation, coordination failures, unvetted capability acquisition, and unbounded self-modification—are fundamentally compositional. Addressing them independently leaves multiplicative failure modes unresolved. By designing four subsystems with explicit composition interfaces, SHARD achieves emergent properties—memory-informed coordination, quality-governed improvement, cross-cutting safety, staleness propagation, and self-healing—that no individual subsystem could produce alone.

Our evaluation validates both individual mechanism correctness and composition properties, while honestly acknowledging gaps: incomplete T4 promotion validation (E3–E4), transitive semantic conflict detection limits (E2), and staleness threshold boundary effects (E6). These gaps define the next phase of research.

The architectural insight generalizes beyond our implementation: persistent autonomous agents require *composed governance*, not independent solutions to independent problems. We release SHARD’s design

as a reference architecture for researchers building persistent agent infrastructure, with the hope that the composition-first approach informs future systems regardless of their specific substrate.

References

- [1] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv:2310.08560*, 2023.
- [2] Q. Wu, G. Banber, Y. Zhang, Y. Wu, B. Li, E. Zhu, et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv:2308.08155*, 2023.
- [3] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv:2305.16291*, 2023.
- [4] J. S. Park, J. C. O’Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein. Generative agents: Interactive simulacra of human behavior. *UIST*, 2023.
- [5] S. Hong, X. Zhuge, J. Chen, X. Zheng, Y. Cheng, et al. MetaGPT: Meta programming for multi-agent collaborative framework. *arXiv:2308.00352*, 2023.
- [6] G. Li, H. A. A. K. Hammoud, H. Itani, D. Khizbullin, and B. Ghanem. CAMEL: Communicative agents for mind exploration of large language model society. *NeurIPS*, 2023.
- [7] Z. Wang, S. Cai, A. Liu, X. Ma, and Y. Liang. DEPS: Describe, explain, plan and select. *arXiv:2302.01560*, 2023.
- [8] Y. Bai, S. Kadavath, S. Kundu, A. Askell, J. Kernion, et al. Constitutional AI: Harmlessness from AI feedback. *arXiv:2212.08073*, 2022.
- [9] A. Madaan, N. Tandon, P. Gupta, S. Halber, L. Gao, et al. Self-Refine: Iterative refinement with self-feedback. *NeurIPS*, 2023.
- [10] N. Shinn, F. Cassano, A. Gopinath, K. R. Narasimhan, and S. Yao. Reflexion: Language agents with verbal reinforcement learning. *NeurIPS*, 2023.